

Multi-Agent RAG Pipeline for Art and Painting Treatises

*Project Status Report: Sub-Project 2, Extending a Local Retrieval-Augmented
Generation System into a Multi-Agent Architecture*

Prepared by Roy Abi Nader
Computer Engineering, Université La Sagesse
August 2026

Table of Contents

Table of Contents.....	2
1. Executive Summary	3
2. Project Overview and Scope	3
3. System Architecture.....	3
3.1 How a Question Flows Through the System	4
4. Component Breakdown	5
4.1 local_rag/: The Retrieval Pipeline	5
4.2 mcp_server/: The Bridge to Everything Else.....	7
4.3 agents/: The Multi-Agent Orchestration Layer.....	7
4.4 frontend/: The Chat Interfaces	9
5. Key Design Decisions.....	10
6. Development Challenges and Resolutions.....	11
6.1 The Retrieval Pipeline (local_rag/).....	11
6.2 The MCP Bridge Layer (mcp_server/)	12
6.3 The Agent Graph (agents/).....	12
6.4 The Frontend and Image Rendering (frontend/)	14
6.5 Cross-Cutting Lessons	15
7. Testing and Evaluation	16
8. Current Project Status	17
9. Known Limitations	17
10. Future Work.....	18
Appendix A: Specialist Agent Reference	19
Appendix B: Project Folder Structure.....	20
Appendix C: Glossary of Terms	21

1. Executive Summary

This report describes the current state of a two-layer artificial intelligence system built entirely with local, free tools. The first layer, referred to as the base pipeline, is a Retrieval-Augmented Generation (RAG) system that answers questions from a corpus of historical painting and drawing treatises. The second layer, built as Sub-Project 2, transforms that single-purpose pipeline into a multi-agent system: a supervisor model reads each incoming question and routes it to one of seven specialist agents, each responsible for a narrow task. Input and output guardrails protect the system on both ends, a persistent chat backend keeps track of multi-turn conversations, and two working browser interfaces let a person actually use the system.

Every part of the system runs on a local machine. No paid application programming interfaces (APIs) are used anywhere, and the language model itself runs through Ollama, a tool for serving large language models locally. This was a firm project constraint from the start, not a limitation discovered along the way.

The remainder of this report walks through the system's architecture, the individual components that make it up, the concrete engineering decisions that were made and why, the real problems encountered while building it and how each was resolved, the current state of the project, and the limitations that remain, stated honestly rather than hidden.

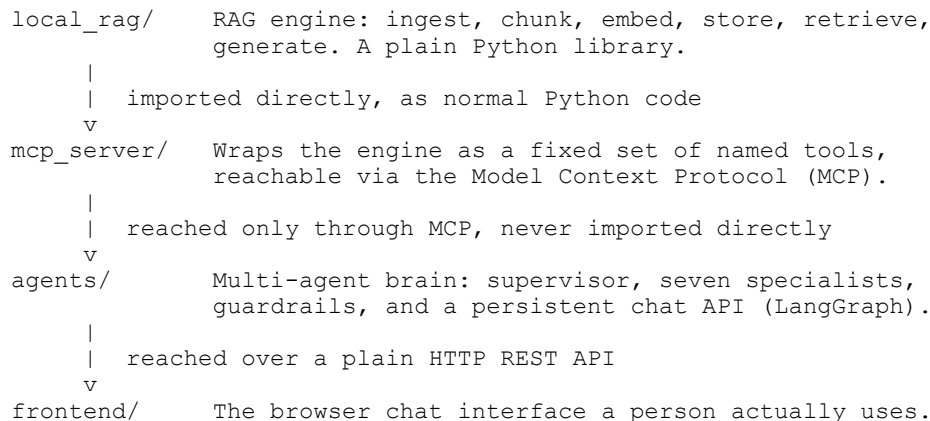
2. Project Overview and Scope

The project began with a base RAG pipeline built as an earlier project. That pipeline ingests a corpus of eleven historical treatises on painting and drawing, most of them scanned books obtained from Archive.org, and turns them into a searchable index that a local language model can draw on to answer questions. After processing, the real corpus amounts to roughly 3,700 pages and about 15,300 text chunks, along with over a thousand extracted images.

Sub-Project 2 took that pipeline and rebuilt the way questions are answered. Instead of a single fixed step of "retrieve, then generate," a supervisor model now looks at each question and decides which of seven specialists should handle it: answering from the corpus directly, listing what documents exist, breaking a compound question into steps, showing relevant images, looking up a specific named painting online, searching for art supplies to buy, or building an invoice from previous searches. Every specialist reaches the underlying pipeline only through a small, well-defined set of tools rather than by importing the pipeline's code directly.

3. System Architecture

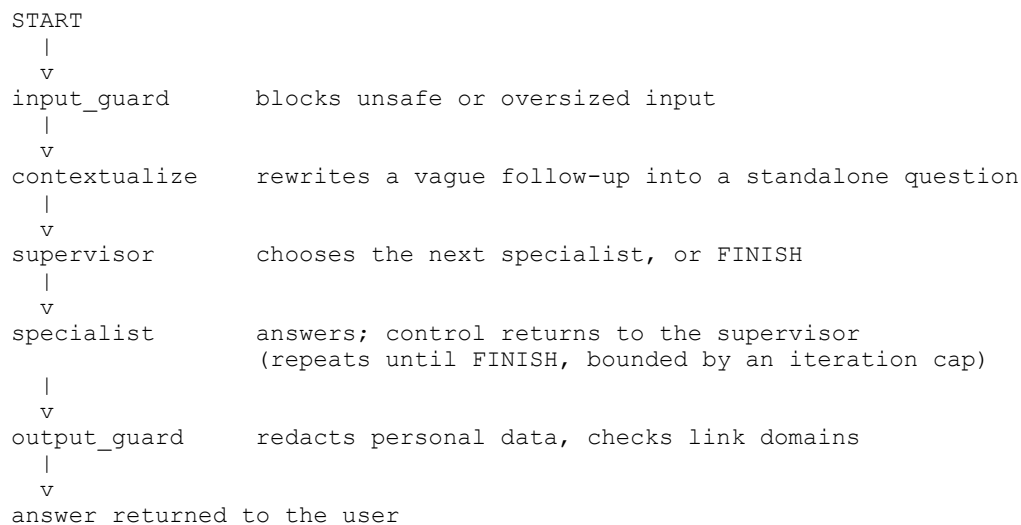
The system is organized into three internal layers, plus a fourth, user-facing layer sitting on top of them. Each layer talks only to the layer directly below it, and the boundary between the agent layer and the retrieval layer is strictly enforced: nothing in the `agents/` folder is allowed to import code from `local_rag/` directly. Every call has to go through the `mcp_server/` layer instead.



This layering is the single most important design decision in the project. Because every specialist can only reach the retrieval pipeline through a documented, named tool, no specialist can quietly do something the tool was not designed for, which makes the whole system easier to audit. It also means the exact same retrieval code can serve more than one kind of client at once: the LangGraph agent described in this report, and any other MCP-compatible tool, such as Claude Code, Cursor, or OpenCode, connecting to the same server independently.

3.1 How a Question Flows Through the System

Once a question reaches the chat backend, it moves through a fixed sequence of steps, implemented as a LangGraph state graph:



The input guard checks the raw question for prompt-injection attempts and excessive length before anything else runs; a flagged question is routed straight to a refusal and never reaches a specialist. The contextualize step rewrites a vague follow-up question into a standalone one using recent conversation history, so that a bare question like "which size is best?" can still be understood on its own. The supervisor is a small language model call that outputs nothing but the name of the next specialist, or the word FINISH; it never writes an answer itself. The chosen specialist runs and returns control to the supervisor, which may route again or finish, bounded by a fixed iteration cap so that a confused supervisor cannot loop forever. Finally, the output guard scans every message produced since the user's last question, not only the final one, for personal information and for links to domains that are not on an approved list, before the answer is returned.

4. Component Breakdown

4.1 local_rag/: The Retrieval Pipeline

This is the oldest and largest part of the project, and everything else is built on top of it. It follows one linear pipeline, ingest, chunk, embed, store, retrieve, generate, and nearly every stage has more than one implementation so that the final choice could be picked by benchmarking real alternatives instead of assuming one was good enough.

Ingestion

Reads raw files, text, PDF, and standalone images, into a common internal format. Because the source documents are scans of physical books rather than born-digital PDFs, a page's text is only trusted if it is genuinely present; otherwise the page is rendered as an image and passed through optical character recognition (OCR) as a fallback. On the real corpus this fallback was not a rare case: two of the eleven source books needed OCR for more than ninety percent of their pages. Ingestion also filters out embedded scanning artifacts (full-page background scans and warped thumbnails), deduplicates near-identical content, tracks which files have already been processed so that re-running ingestion only touches new or changed files, and automatically captions extracted images using a vision-language model.

Chunking

Splits ingested text into pieces small enough to embed and retrieve accurately. Six methods were implemented and benchmarked against one another on the real corpus: fixed-size, recursive, sentence-based, semantic (topic-shift-based), structure-aware, and parent-child. Parent-child chunking, which pairs small child chunks used for matching with larger parent chunks used for generating the final answer, was selected for production because it gave the most consistent results and the best balance between retrieval precision and answer context.

Embeddings

Converts text, and separately images, into numeric vectors. Three backends were built: Hugging Face (running in-process), Ollama (over the network), and CLIP, which is the only one of the three that places images and text in the same vector space, so a text question can retrieve a relevant image directly. Of five candidate text embedding models compared, all-MiniLM-L6-v2 was selected for production: it was the fastest of the models that worked correctly, and it produced the cleanest separation between related and unrelated text in a quick sanity check performed before any real corpus data was touched.

Vector Storage

Stores every vector along with its text and metadata and answers similarity searches. Two backends were compared, ChromaDB (embedded, requiring no separate server) and Qdrant (a separate client-server system). Chroma was selected for production: queries ran roughly nine times faster than Qdrant's in a single-machine setup, even though Qdrant's write throughput was about twice as fast.

Retrieval

Given a question, finds the chunks most likely to contain the answer. Six strategies were implemented: plain vector search, hybrid search (vector search combined with keyword search), a cross-encoder reranker, multi-query expansion, a rule-based query router, and contextual compression, along with a separate image retriever built on CLIP. The two strategies that sounded more sophisticated, hybrid search and multi-query expansion, both scored worse than plain vector search on precision and recall across two separate labeled question sets, because this corpus's real questions are mostly abstract and conceptual, and blending in keyword matching added noise rather than useful signal. Plain vector search was selected for production.

Generation

Turns retrieved chunks and a question into a final answer, using a local model through Ollama or Hugging Face as a fallback. Of several candidate models compared, three completed a full evaluation (llama3.2, mistral, and phi3), and llama3.2 was selected as the production generator for having the best faithfulness per second among the models that finished evaluation. A dual-modality mode also exists, which drafts a text answer and an image answer independently and combines them.

Safety, Evaluation, and Supporting Modules

The safety/ module handles prompt-injection detection and personal-information redaction for ingested content, and later grew a domain allowlist once the multi-agent layer added tools that reach the internet. The evaluation/ module supplies labeled ground truth, including a local-model-

judge evaluator so retrieval quality can be measured without relying on a paid evaluation API. Four further folders, covering small local model comparisons, quantization for weaker hardware, vision-language model comparisons, and an alternative serving backend, were built and benchmarked but deliberately left unwired from the day-to-day pipeline to protect the project timeline, not because they failed.

Four entry points expose this pipeline: `config.py`, the single source of truth for every path and model name used anywhere in the project; `pipeline.py`, a command-line tool for ingesting documents and asking questions; `stages.py`, which runs each pipeline stage as an independent, checkpointed step, useful for debugging one stage without redoing everything before it; and `api.py`, a REST wrapper around the same two core operations.

4.2 mcp_server/: The Bridge to Everything Else

This folder's job is to translate `local_rag/`'s Python functions into a standard set of tools that any compatible client can call, using the Model Context Protocol (MCP). It began with two tools, `retrieve` and `generate_answer`, and one resource listing the ingested documents. Four more tools were added later: `retrieve_images`, `search_painting_online`, `search_art_supplies`, and `generate_invoice`.

`web_tools.py` backs the two internet-facing tools, using Wikipedia's public interface and a free, keyless general web search library for painting lookups, and a listings search restricted to Amazon and eBay for art supplies, both filtered against a small, hand-approved list of domains. `image_tools.py` backs image retrieval, pulling results from the CLIP-indexed image store along with their captions, and was later extended to return the actual image bytes, base64-encoded, directly inside the tool response. `invoice_tools.py` performs all invoice arithmetic in plain Python, deliberately never handing a total or a line calculation to a language model.

Every tool in this layer follows one rule: if something it depends on is not available, no internet connection, no ingested corpus, a missing library, it returns an empty result rather than raising an exception. An empty list is treated as a legitimate answer, "nothing found right now," not an error condition. This layer is covered by offline smoke tests that fake the network and file system entirely, and by a separate script that confirms a single running server instance can correctly serve two different kinds of client, an IDE-style tool caller and a LangGraph agent, at the same time.

4.3 agents/: The Multi-Agent Orchestration Layer

This is where a question is actually handled from start to finish: checked, routed, answered, and checked again before being shown to the user. It is built with LangGraph, a library for building applications as an explicit graph of steps.

Shared State and Prompts

state.py defines the single data structure passed between every node in the graph: the running conversation, the current routing decision, an iteration counter, and two guardrail-only fields recording whether a turn was blocked and why. It was deliberately kept small and stable; nearly every new feature added over the life of the project was added as a new node or file rather than a change to this shared structure. prompts.py holds every system prompt used anywhere in the graph, one narrow prompt per specialist plus the supervisor's own routing prompt, kept in one place so prompt wording could be iterated on without touching any wiring code.

The Seven Specialists

specialists.py defines seven specialist agents, each with a narrow job and a deliberately different tool footprint, so the project demonstrates several distinct routing patterns rather than one generic agent trying to do everything. A full reference table for all seven appears in Appendix A.

- **retrieval_qa:** a genuine reasoning loop that can call the retrieve tool more than once before generating a final, cited answer.
- **corpus_meta:** answers questions about the corpus itself, such as document counts or titles, with zero tools at all. The document list is baked directly into its prompt, so it is structurally unable to invent anything about a document's content.
- **multi_hop:** handles compound questions with a fixed, three-step script, decompose, retrieve twice, and combine, rather than an open-ended loop, so its cost per question is predictable.
- **image_qa:** shows real corpus images with their captions, making zero language model calls; it purely formats what the image tool already returned.
- **painting_lookup:** answers a question about one specific named painting by combining a local corpus search with a live web lookup.
- **product_search:** always searches the internet, never the corpus, since the corpus has no product data, and splits results into beginner and professional tiers.
- **invoice:** reads back through the entire conversation for earlier product_search results and totals them using plain Python arithmetic, with the language model used for wording only.

The Supervisor

supervisor.py decides, on every visit, which specialist should handle a question, or whether the turn is finished. It never writes an answer itself; its model is constrained to output only a route name. This file carries the largest share of the project's safety engineering, described in detail in Section 6.3, because a small local model turned out to be considerably less reliable at following plain routing instructions than the prompt alone assumed.

Guardrails and Context Handling

`guardrails.py` implements the input guard and the output guard described in Section 3.1. Neither one calls a language model; both are fast, deterministic checks over text, on the reasoning that a guardrail depending on a model's cooperation is a weaker guardrail. `contextualize.py` rewrites an ambiguous follow-up question into a standalone one using recent conversation history, running once per turn immediately after the input guard, and is skipped entirely, with zero model calls, on a conversation's very first message.

Wiring, the Chat API, and Testing

`graph.py` wires every node above into one compiled `LangGraph` graph. It builds its specialist routing map generically from whatever `specialists.py` returns rather than a hardcoded list, which is what allowed four additional specialists to be added later with no change to this file at all. `api.py` is a FastAPI server that turns the underlying one-shot graph into a real, multi-turn chatbot: the graph, the MCP client, and a SQLite-backed `LangGraph` checkpointer are all built once at process startup and reused for every request, rather than rebuilt per call, and conversation history persists across requests and server restarts, keyed by a conversation thread identifier. The API also exposes endpoints for browsing past conversations, listing which specialist names may be forced for isolated testing, and serving corpus images to a browser.

`eval_phase5.py` runs a fixed set of ten designed questions, covering single-specialist routing, multi-step questions, out-of-scope questions, and adversarial input, through the real compiled graph, and produces a results table. It deliberately grades only routing correctness mechanically, and marks answer-quality rows for a human reader to judge rather than pretending that step can be automated. `agent_mcp_server.py` exposes the entire guarded graph as one further MCP tool, so an external client can get a fully-guarded answer with a single call instead of reaching the raw retrieval tools directly. Nine test files cover every module above, each following the same two-level pattern: fast offline tests using a scripted fake model and fake tool responses, and a separate live-run option for confirming behavior against a real Ollama model and the real corpus.

4.4 frontend/: The Chat Interfaces

Two working interfaces exist on top of the chat API, both talking to the same backend contract. `agents/static/chat.html` is a single, self-contained HTML and JavaScript page with no build step, served directly by the FastAPI backend, useful for trying the system with zero setup. `frontend/` is a proper React application built with Vite and the `assistant-ui` component library, chosen because it already solves message rendering, streaming state, and input composition, leaving this project to supply only where the messages actually come from.

The one significant design decision in this layer is the choice of runtime. `assistant-ui`'s default runtime assumes the browser itself owns the conversation history, which is backwards for this

project, since the FastAPI backend already owns the real history through its checkpoint, and the invoice specialist specifically depends on that history being real and server-owned. The React app therefore uses assistant-ui's external store runtime instead: it is handed a plain array of messages that the application controls, seeded from the backend's history endpoint on page load and appended to after each response, rather than reimplementing a client-side history format only to bolt a backend fetch onto it.

Beyond the core chat view, the React application includes a sidebar for browsing and switching between past conversations, a dropdown for forcing a single specialist to answer the next message directly for testing purposes, and a markdown renderer for assistant messages that correctly displays retrieved images, complete with a client-side download button for each one. The details of how image rendering was made to work correctly are described in Section 6.4.

5. Key Design Decisions

Every major decision in this project, from which chunking method to use to how routing should be enforced, was made by testing real alternatives against the real corpus or by responding to a real observed failure, rather than by assumption. The table below summarizes the most significant of these decisions; the reasoning behind each is expanded further in Sections 4 and 6.

Decision Area	Options Evaluated	Selection	Basis for the Decision
Chunking strategy	6 methods (fixed-size, recursive, sentence, semantic, structure-aware, parent-child)	Parent-child	Most consistent results; balances retrieval precision with generation context
Text embedding model	5 candidate models across 3 backends	all-MiniLM-L6-v2	Fastest working candidate; cleanest similarity separation in a pre-run sanity check
Vector store	ChromaDB vs. Qdrant	ChromaDB	Roughly 9x faster query time in a single-machine setup
Retrieval strategy	6 strategies including hybrid and multi-query	Plain vector search	Outperformed hybrid and multi-query on precision and recall for this corpus
Generation model	5 candidates; 3 completed evaluation	llama3.2	Best faithfulness per second among models that completed evaluation

Decision Area	Options Evaluated	Selection	Basis for the Decision
Routing enforcement	Prompt-only vs. code-level checks	Code-level checks	A small local model proved unreliable at following plain routing instructions
Monetary and link handling	Model-computed vs. Python-computed	Python-computed	A model relaying a number or a link was an identified, avoidable failure risk

6. Development Challenges and Resolutions

Building this project surfaced two broad categories of problem: issues that could be reasoned about and fixed before anything ran, and issues that only appeared once the system was actually exercised against a real local model, a real corpus, and, later, a real internet connection. Both kinds are documented here, organized by the layer in which they occurred, because both are a normal and expected part of connecting several real systems together.

6.1 The Retrieval Pipeline (`local_rag/`)

- Chunk metadata was silently being dropped somewhere between chunking and storage, so a retrieved chunk could not be traced back to its source file. This was found by tracing the real prompt and data path end to end rather than by a unit test.
- ChromaDB enforces a maximum batch size for a single insert call; ingesting the full corpus in one call failed outright once the real chunk count exceeded that limit, and had to be batched automatically using the limit Chroma's own client reports.
- Re-running ingestion on the same folder caused manifest collisions in the incremental-ingest tracking, fixed by hashing each chunk's actual content into its identifier, so the same content always produces the same identifier no matter how many times it is re-ingested.
- A system-level HTTP proxy was silently intercepting local network traffic intended for Ollama, causing local model calls to fail in a confusing way with no obvious error. This was fixed by explicitly disabling proxy use for local requests.
- The structure-aware chunking method produced one page that generated a single chunk of 85,938 words, far outside the normal range, traced to a page-extraction problem rather than a flaw in the chunking rule itself, and left as a documented, unresolved limitation rather than silently ignored.
- One candidate embedding model, `mxbai-embed-large`, failed outright on the real corpus because at least one chunk's OCR-derived text exceeded the model's context window, and

because the run was wrapped in a single error handler rather than one per chunk, that one oversized chunk discarded an entire run's progress.

- A naive image extractor initially pulled roughly four images per page, most of them scanning artifacts rather than real illustrations. Filtering by an image's own pixel dimensions caught most of these but missed low-resolution scans and warped thumbnails. The fix that actually worked was to check where an image sits on the page instead: an image covering nearly the whole page is treated as a background scan regardless of its resolution.
- Hybrid retrieval was roughly one hundred times slower per question than expected, because its keyword index was being rebuilt from scratch on every call rather than cached. After that performance bug was fixed, hybrid retrieval still scored worse than plain vector search on accuracy for this corpus's mostly abstract, conceptual questions.

6.2 The MCP Bridge Layer (mcp_server/)

- The server needed to resolve its own module import path correctly regardless of which working directory it was launched from, since different callers, Claude Code, Cursor, and the LangGraph agent, do not all set that up the same way.
- MCP communicates over standard input and output as JSON-RPC, so anything else printed to standard output, a stray print statement or a noisy library, corrupts that channel. All such output had to be redirected to standard error instead.
- A Windows-specific fix was needed for how the client locates the Python interpreter to launch the server subprocess with, since Windows resolves this differently from Linux and macOS.
- The library connecting the agent to the MCP server wraps every tool result in a content-block envelope rather than returning the raw value, which required unwrapping with a JSON parse step and a fallback for tools that return a plain string instead of JSON.
- Looking up a named painting online initially resolved to the wrong result entirely: a question about "the Mona Lisa" matched a 2003 film sharing similar words, because the full question, including wrapper phrasing like "tell me about," was passed directly into the search. This was fixed by stripping that kind of question phrasing before the lookup, at the tool level, so every future caller benefits automatically.

6.3 The Agent Graph (agents/)

- A small local language model does not reliably follow plain-text instructions. The routing prompt explicitly told the model never to re-route to a specialist that had already answered, and never to finish before anything had answered at all. Live runs showed the model doing both anyway: on one run it picked the same specialist on every single visit, and on another

its very first output was FINISH with nothing having run yet, which would have silently returned no answer at all. The fix was to stop relying on the prompt for correctness and check the model's output in code instead: a schema check, a check that the route name is a real specialist, a check that it is not repeating an already-tried specialist, and a check that it is not finishing before anything has run.

- retrieval_qa's answers initially had no source citations, even though its prompt required them. A prompt-only fix did not help, because the model was re-narrating the already-cited tool output in its own words and dropping the citations in the process. The actual fix pulled the tool's original, correctly cited output directly from the message history instead of trusting the model's own restatement of it, a case where a design failure had initially been mistaken for a wording failure.
- When the repeat-route guard above forced a turn to finish, the guard would sometimes walk all the way to a specialist with nothing useful to say, and the code that selects the final answer, by simply taking whatever came last, would show that irrelevant response to the user even though an earlier specialist had already answered correctly. The fix was to reaffirm the first specialist's real answer as the final message whenever a finish is forced rather than genuinely chosen by the model.
- The output guard originally scanned only the last message in the conversation for personal information and unapproved links. This was confirmed live to be insufficient: once the supervisor began appending a short note after a forced finish, that note, not the real answer, became the last message, meaning a leak in the real answer could have passed through undetected. The fix was to scan every message produced since the user's last turn, not only the final one.
- The supervisor's system prompt was originally rebuilt with the full conversation transcript folded into it on every call. The small local model conditioned much more reliably on information placed in the human turn instead, the same pattern the multi_hop specialist already used successfully, so the system prompt was made fully static and a separate template now carries the transcript in the user turn.
- As the specialist pool grew from three to seven, the iteration cap that limits how many times the supervisor may loop on one question was deliberately raised from four to eight, large enough for the fallback logic to reach every specialist in a genuine worst case, but still small enough that a confused model cannot burn an unbounded number of calls.
- Every specialist and the supervisor's routing decision look only at the single newest message, which is correct within one turn but leaves a real gap across turns: a bare follow-up such as "which size is best?" right after a question about brushes carries no retrievable content of its own. The contextualize node was added specifically to close this gap, rewriting a vague follow-up into a standalone question using recent history, in exactly one place in the pipeline so every specialist benefits automatically.

- The graph's original one-shot convenience function rebuilds an entire graph, including a fresh MCP server subprocess, on every single call, and starts with a completely empty message list every time. This is fine for a script asking one question and exiting, but wrong for a real chatbot on two counts: every message would pay several seconds of subprocess startup cost, and the invoice specialist would be unable to see anything from earlier in the conversation no matter how the question was phrased. The fix was a persistent FastAPI process that builds the graph once and reuses it, backed by a real, SQLite-based checkpointer keyed by a conversation identifier.
- Once conversations became persistent, four state fields, the current route, the iteration counter, whether the turn was blocked, and which injection patterns matched, turned out to have no defined rule for combining an old value with a new one, meaning the persistence layer's default behavior would silently carry the previous turn's leftover value into the next turn. This was caught before shipping by testing the exact shape of the problem on a small, throwaway version of the graph, and fixed by having every new turn explicitly reset those specific fields to their starting values.
- Letting a person browse past conversations needed a way to list every stored conversation, but the underlying persistence library has no built-in method for that; it only knows how to fetch one conversation once its identifier is already known. The fix was to read directly from the same underlying database table the persistence layer already uses internally, rather than building and maintaining a second, separate index that could drift out of sync.
- A feature allowing a single specialist to be forced for isolated testing needed to skip the supervisor entirely for one turn without disturbing normal behavior for everyone else. The fix reused the same principle established for invalid model output: an unrecognized or missing forced choice degrades cleanly to the normal supervisor-driven behavior, confirmed with a dedicated test proving the supervisor is called zero times when a specialist is forced, and exactly as many times as before when it is not.

6.4 The Frontend and Image Rendering (frontend/)

- assistant-ui's setup tool normally fetches a prebuilt, polished chat component from a remote registry, which was not reachable under this project's network restrictions. The underlying building blocks were assembled by hand instead, and the whole application was type-checked against the real installed library before being handed over, specifically to avoid shipping code that looked correct but would not actually build.
- Retrieved images were originally returned as a raw local filesystem path, which is meaningless to a browser that cannot reach the server's own disk. This was first fixed by adding an endpoint that serves an image by its filename only, with explicit protection against a filename attempting to escape its intended folder, and later improved further by embedding the image's actual bytes as base64 data directly inside the tool response, so no

separate serving step is required at all. The original path-based approach was kept as a fallback rather than removed.

- Neither chat interface originally rendered markdown at all; a returned image would display as literal characters such as an exclamation mark and brackets rather than a picture. The simple HTML page was given a small, deliberately narrow renderer that escapes all text first and then recognizes only three safe patterns, images, links, and a caption's italics line, to keep the surface area for injection small. The React application had no markdown-rendering package installed, and one was added and wired in specifically for assistant messages, leaving user-typed messages as plain text.
- The two interfaces are served from two different origins, the simple HTML page from the same backend that serves images, and the React application from its own separate development server, so a relative image address that worked correctly for one would silently fail for the other. This was fixed with a small component on the React side that rewrites a relative image address onto the backend's own address before handing it to the browser.
- A stray closing bracket character inside a freely generated image caption, since vision-language-model captions carry no punctuation guarantee, could prematurely close the markdown image syntax, causing an entire base64 payload, sometimes hundreds of kilobytes, to spill into the chat as visible text instead of being parsed as an image. This was fixed by escaping markdown-special characters in every caption before it is built into an image tag.
- After that fix, images still failed to render, this time because the markdown-rendering library blocks embedded data addresses from appearing in an image's source attribute by default, as a general security precaution. The fix was a narrowly scoped override that allows a base64 image data address through only for an actual image element's source attribute, leaving every other case, links and non-image data addresses, going through the original, safe default untouched.
- A search tool for art supplies was silently returning nothing, with only a generic message about the search backend possibly being unavailable. The underlying search library was found to default to trying every registered search engine, including two that can never return a shopping listing for a site-restricted search, and with this project's small result-count settings those two engines could occupy the very first search attempts before a real general-purpose engine was ever tried. The fix was to explicitly exclude those two engines from this particular tool.

6.5 Cross-Cutting Lessons

A small number of themes repeated across nearly every phase of the project rather than showing up once:

- Offline tests proved the code was correctly wired, not that a real model would behave as assumed. Nearly every genuine bug described above, missing citations, the wrong Wikipedia result, the repeat-route guard surfacing the wrong final answer, the output guard's scanning gap, was found only by actually running the system live against a real model and real data, never by an offline test alone.
- A small local language model cannot be trusted to simply follow plain-text instructions. Every real fix arising from a model's misbehavior took the same shape: a structural, code-level check, such as a membership test or a repeat-route guard, rather than an additional sentence in a prompt.
- Windows contributed its own category of problem, both in how the MCP subprocess is spawned and in how a system proxy silently intercepted local traffic, neither of which would have been obvious from reading the code alone.
- Nearly every extension to the project was additive rather than a rewrite. The shared state structure and the graph-wiring file both changed remarkably little across many phases of work, because new capability was consistently added as a new node or a new file rather than by restructuring what already existed.
- The project repeatedly found that a simpler method won once tested against real data: plain vector search beat both hybrid search and multi-query expansion, and a single small local model handled routing well enough once backed by code-level checks, without needing a larger or more expensive model.

7. Testing and Evaluation

Testing in this project follows a consistent two-tier pattern at every layer. Fast, offline smoke tests substitute a scripted fake language model and fake tool responses, so they run in seconds with no dependency on a running Ollama server or a live internet connection; the MCP bridge layer has its own such tests, and the agent layer has nine separate test files covering every module. These tests are effective at proving the wiring is correct, that a message ends up with the right sender, that a tool is called the expected number of times, that a malformed response does not crash anything, but they cannot predict how a real small model behaves under repetition or how a real search engine ranks a real query.

A second tier of live-run scripts exists specifically to close that gap: they run the real specialists against a real Ollama model and the real ingested corpus, and are meant to be run once the offline tests already pass, as a check that the answers produced are actually sensible rather than merely non-crashing.

`eval_phase5.py` adds a structured evaluation harness on top of both tiers: a fixed set of ten designed questions, covering single-specialist routing, multi-step questions, out-of-scope questions, and

adversarial input, is run through the fully compiled graph, and the results are written to a report-ready table. This harness deliberately grades only whether the supervisor's routing decision matched what was expected; whether the final answer text is actually correct is left to a human reader, and the results table marks clearly which rows are a mechanical pass or fail and which still need to be read by a person.

At the time of this report, the four newer specialists, `image_qa`, `painting_lookup`, `product_search`, and `invoice`, have full offline smoke-test coverage but have not yet completed a full pass through the `eval_phase5.py` scoring table the way the original three specialists have. This is recorded as an open item in Section 10.

8. Current Project Status

- **local_rag/**: complete and benchmarked; every production configuration choice is backed by a benchmark run against the real corpus.
- **mcp_server/**: all six tools are built, covered by offline tests, and confirmed working with two different kinds of client connected at once.
- **agents/**: all seven specialists, the validated-routing supervisor, both guardrails, the contextualize step, the persistent chat API, and the evaluation harness are built and passing their tests, with several pieces additionally confirmed against real live runs.
- **frontend/**: both the simple HTML chat page and the React application are functional, including real image display with a download option, conversation history browsing, and a forced-specialist testing mode.

Taken together, the system is functionally complete from document ingestion through a working browser chat interface, and its development history is documented as a series of real, observed failures and fixes rather than presented as having worked correctly on the first attempt.

9. Known Limitations

The following limitations are stated deliberately, as documented tradeoffs made under the project's own constraints, no paid APIs, a small local model handling routing, and limited project time, rather than as gaps discovered by accident.

- Prices returned by the art-supply search are a best-effort reading of a search-result snippet, not a live pricing lookup, and can be stale or missing; a real product pricing API would fix this but requires a paid developer key.

- The domain allowlist used for internet links is a small, hand-picked list of trustworthy sites, not a live reputation or trust-scoring system; it guarantees that a domain is pre-approved, which is a weaker guarantee than that any specific page on it is accurate.
- Personal-information redaction in the output guard only catches structured patterns, such as email addresses, phone numbers, and card numbers; a name or address written in ordinary prose would not be caught.
- There is no streaming anywhere in the chat layer; every answer arrives as a single block once the supervisor loop finishes, not token by token.
- There is no authentication in front of the chat API.

10. Future Work

- Complete a full live evaluation pass for the four newer specialists through the `eval_phase5.py` scoring table, matching the coverage already in place for the original three.
- Add streaming responses to the chat layer so answers appear progressively rather than as a single block.
- Add authentication and proper multi-user support to the chat API.
- Revisit the optional, deeper items from the original project specification that have not yet been started: human-in-the-loop interrupts, containerizing the MCP server behind HTTP with authentication in front of it, an LLM-based input classifier layered on top of the current pattern-matching guard, and rebuilding one piece of the system in a different framework for comparison.

Appendix A: Specialist Agent Reference

A consolidated reference for the seven specialist agents defined in `agents/specialists.py`, expanding on the summary given in Section 4.3.

Specialist	Tool Footprint	Model Calls	Purpose
retrieval_qa	retrieve, generate_answer (repeatable loop)	Yes, iterative	Answers a single-topic question from the corpus, retrieving as many times as it judges necessary.
corpus_meta	None; static prompt	Yes, one call	Answers questions about the corpus itself, such as document counts or titles.
multi_hop	retrieve, called twice (fixed script)	Yes, one call	Splits a compound question into two parts, retrieves for each, and combines the answers.
image_qa	retrieve_images	No	Formats retrieved corpus images and captions with no model call.
painting_lookup	retrieve, search_painting_online	Yes, one call	Combines a corpus search with a live web and Wikipedia lookup for one named painting.
product_search	search_art_supplies	Yes, one call	Searches the internet for art supplies and splits results into beginner and professional tiers.
invoice	generate_invoice	No	Reads prior product_search results from the conversation and totals them in plain arithmetic.

Appendix B: Project Folder Structure

```
project-root/
|-- local_rag/
|   |-- ingestion/, chunking/, embeddings/, vectorstore/,
|   |-- retrieval/, generation/, safety/, evaluation/
|   |-- slm/, quantization/, vlm/, serving/ (benchmarking only)
|   |-- config.py, pipeline.py, stages.py, api.py
|
|-- mcp_server/
|   |-- server.py, image_tools.py, web_tools.py, invoice_tools.py
|   |-- test_new_tools_smoke.py, test_langgraph_client.py
|
|-- agents/
|   |-- state.py, prompts.py, mcp_client.py
|   |-- specialists.py, supervisor.py, contextualize.py
|   |-- guardrails.py, graph.py, api.py
|   |-- eval_phase5.py, agent_mcp_server.py
|   |-- static/chat.html
|   |-- test_*.py (nine files)
|
|-- frontend/
|   |-- src/api.ts, runtime.ts, types.ts
|   |-- src/components/Thread.tsx, Message.tsx, Composer.tsx,
|       ChatHistory.tsx, ToolSelector.tsx, MarkdownText.tsx
```

Appendix C: Glossary of Terms

- **RAG (Retrieval-Augmented Generation):** a technique where a language model's answer is grounded in passages retrieved from a document collection, rather than generated purely from what the model already knows.
- **MCP (Model Context Protocol):** a standard protocol that lets a language-model client discover and call a fixed set of named tools without importing the underlying code directly.
- **LangGraph:** a library for building an application as an explicit graph of steps, where state is passed between nodes and edges decide what runs next.
- **Supervisor pattern:** a design in which one model or component decides which of several specialized components should handle a given task, rather than one component trying to do everything.
- **Checkpointing:** persisting a LangGraph's state to storage, keyed by a conversation identifier, so a conversation can resume correctly across separate requests or a server restart.
- **CLIP:** a model that places images and text in the same vector space, allowing a text query to retrieve a relevant image directly.
- **OCR (Optical Character Recognition):** the process of extracting text from a scanned image of a page.
- **BM25:** a classical keyword-ranking algorithm used for text search, often combined with vector search in a hybrid retrieval strategy.
- **Guardrail:** a deterministic check, not dependent on a model's cooperation, applied to input before it is processed or to output before it is shown to a user.
- **Smoke test:** a fast, offline test using faked dependencies, intended to catch wiring and logic errors without needing a real model or a real network connection.